

Lab Practical #10:

1. Implementation of DES Algorithm.

Program:

```
// Implementation of DES Algorithm.

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define SIZE 100

/* ===== HEX ⇔ BIN ===== */

void hex2bin(char *hex, char *bin)
{
    char *map[16] = {
        "0000", "0001", "0010", "0011",
        "0100", "0101", "0110", "0111",
        "1000", "1001", "1010", "1011",
        "1100", "1101", "1110", "1111"
    };
    bin[0] = '\0';
    for(int i=0; hex[i]!='\0'; i++)
    {
        if(hex[i]>='0' && hex[i]<='9')
            strcat(bin, map[hex[i]-'0']);
        else
            strcat(bin, map[hex[i]-'A'+10]);
    }
}

void bin2hex(char *bin, char *hex)
{
    char *map[16] = {"0", "1", "2", "3", "4", "5", "6", "7",
        "8", "9", "A", "B", "C", "D", "E", "F"};
    hex[0] = '\0';
    for(int i=0; i<64; i+=4)
    {
        int num=0;
        for(int j=0; j<4; j++)
            num = num*2 + (bin[i+j]-'0');
        strcat(hex, map[num]);
    }
}
```

```
void xor(char *a, char *b, char *res)
{
    for(int i=0;a[i]!='\0';i++)
        res[i]=(a[i]==b[i])?'0':'1';
    res[strlen(a)]='\0';
}

void shift_left(char *key,int shifts)
{
    while(shifts--)
    {
        char first=key[0];
        for(int i=0;i<strlen(key)-1;i++)
            key[i]=key[i+1];
        key[strlen(key)-1]=first;
    }
}

void permute(char *in,int *table,int n,char *out)
{
    for(int i=0;i<n;i++)
        out[i]=in[table[i]-1];
    out[n]='\0';
}

/* ===== DES TABLES ===== */

/* Initial Permutation */
int initial_perm[64]={58,50,42,34,26,18,10,2,
60,52,44,36,28,20,12,4,
62,54,46,38,30,22,14,6,
64,56,48,40,32,24,16,8,
57,49,41,33,25,17,9,1,
59,51,43,35,27,19,11,3,
61,53,45,37,29,21,13,5,
63,55,47,39,31,23,15,7};

/* Final Permutation */
int final_perm[64]={40,8,48,16,56,24,64,32,
39,7,47,15,55,23,63,31,
38,6,46,14,54,22,62,30,
37,5,45,13,53,21,61,29,
36,4,44,12,52,20,60,28,
35,3,43,11,51,19,59,27,
34,2,42,10,50,18,58,26,
```



Date: 12/03/2026

```
33,1,41,9,49,17,57,25});

/* Expansion Table */
int exp_d[48]={32,1,2,3,4,5,4,5,
6,7,8,9,8,9,10,11,
12,13,12,13,14,15,16,17,
16,17,18,19,20,21,20,21,
22,23,24,25,24,25,26,27,
28,29,28,29,30,31,32,1};

/* Straight Permutation */
int per[32]={16,7,20,21,
29,12,28,17,
1,15,23,26,
5,18,31,10,
2,8,24,14,
32,27,3,9,
19,13,30,6,
22,11,4,25};

/* PC-1 */
int keyp[56]={57,49,41,33,25,17,9,
1,58,50,42,34,26,18,
10,2,59,51,43,35,27,
19,11,3,60,52,44,36,
63,55,47,39,31,23,15,
7,62,54,46,38,30,22,
14,6,61,53,45,37,29,
21,13,5,28,20,12,4};

/* PC-2 */
int key_comp[48]={14,17,11,24,1,5,
3,28,15,6,21,10,
23,19,12,4,26,8,
16,7,27,20,13,2,
41,52,31,37,47,55,
30,40,51,45,33,48,
44,49,39,56,34,53,
46,42,50,36,29,32};

/* Shift table */
int shift_table[16]={1,1,2,2,2,2,2,2,
1,2,2,2,2,2,2,1};

/* S-Box */
int sbox[8][4][16]={
```

Date: 12/03/2026

```
{ {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
  {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
  {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
  {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}},

{ {15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
  {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
  {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
  {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}},

{ {10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
  {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
  {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
  {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}},

{ {7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
  {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
  {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
  {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}},

{ {2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
  {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
  {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
  {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}},

{ {12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
  {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
  {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
  {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}},

{ {4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
  {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
  {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
  {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}},

{ {13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
  {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
  {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
  {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}}
};

/* ===== MAIN ===== */

/* ===== MAIN ===== */

int main()
```

```
{
    char pt[20], key[20];
    char pt_bin[100], key_bin[100];

    printf("Enter 16 Hex Plaintext: ");
    scanf("%s",pt);

    printf("Enter 16 Hex Key: ");
    scanf("%s",key);

    printf("\nOriginal Plaintext: %s\n", pt);

    hex2bin(pt,pt_bin);
    hex2bin(key,key_bin);

    /* ---- Key Generation ---- */
    char key56[100];
    permute(key_bin,keyp,56,key56);

    char left[30],right[30];
    strncpy(left,key56,28); left[28]='\0';
    strncpy(right,key56+28,28); right[28]='\0';

    char round_key[16][50];

    for(int i=0;i<16;i++)
    {
        shift_left(left,shift_table[i]);
        shift_left(right,shift_table[i]);

        char combine[60];
        strcpy(combine,left);
        strcat(combine,right);

        permute(combine,key_comp,48,round_key[i]);
    }

    /* ===== ENCRYPTION ===== */

    char ip[100];
    permute(pt_bin,initial_perm,64,ip);

    char L[40],R[40];
    strncpy(L,ip,32); L[32]='\0';
    strncpy(R,ip+32,32); R[32]='\0';
```

```
for(int i=0;i<16;i++)
{
    char R_exp[60];
    permute(R,exp_d,48,R_exp);

    char xored[60];
    xor(R_exp,round_key[i],xored);

    char sbbox_str[40]="";

    for(int j=0;j<8;j++)
    {
        int row = (xored[j*6]-'0')*2 + (xored[j*6+5]-'0');
        int col = (xored[j*6+1]-'0')*8 +
                  (xored[j*6+2]-'0')*4 +
                  (xored[j*6+3]-'0')*2 +
                  (xored[j*6+4]-'0');

        int val = sbbox[j][row][col];

        char bin[5];
        for(int k=3;k>=0;k--)
            bin[3-k] = ((val>>k)&1)+'0';

        bin[4]='\0';
        strcat(sbbox_str,bin);
    }

    char f_out[40];
    permute(sbbox_str,per,32,f_out);

    char newR[40];
    xor(L,f_out,newR);

    strcpy(L,R);
    strcpy(R,newR);
}

char combine[100];
strcpy(combine,R);
strcat(combine,L);

char cipher_bin[100];
permute(combine,final_perm,64,cipher_bin);

char cipher_hex[20];
```

Date: 12/03/2026

```
bin2hex(cipher_bin,cipher_hex);

printf("\nEncrypted Cipher Text: %s\n",cipher_hex);

/* ===== DECRYPTION ===== */

char cipher_bin_input[100];
hex2bin(cipher_hex,cipher_bin_input);

char ip_dec[100];
permute(cipher_bin_input,initial_perm,64,ip_dec);

char Ld[40],Rd[40];
strncpy(Ld,ip_dec,32); Ld[32]='\0';
strncpy(Rd,ip_dec+32,32); Rd[32]='\0';

for(int i=15;i>=0;i--)
{
    char R_exp[60];
    permute(Rd,exp_d,48,R_exp);

    char xored[60];
    xor(R_exp,round_key[i],xored);

    char sbbox_str[40]="";

    for(int j=0;j<8;j++)
    {
        int row = (xored[j*6]-'0')*2 + (xored[j*6+5]-'0');
        int col = (xored[j*6+1]-'0')*8 +
                  (xored[j*6+2]-'0')*4 +
                  (xored[j*6+3]-'0')*2 +
                  (xored[j*6+4]-'0');

        int val = sbbox[j][row][col];

        char bin[5];
        for(int k=3;k>=0;k--)
            bin[3-k] = ((val>>k)&1)+'0';

        bin[4]='\0';
        strcat(sbbox_str,bin);
    }

    char f_out[40];
    permute(sbbox_str,per,32,f_out);
```

Date: 12/03/2026

```
char newR[40];
xor(Ld,f_out,newR);

strcpy(Ld,Rd);
strcpy(Rd,newR);
}

char combine_dec[100];
strcpy(combine_dec,Rd);
strcat(combine_dec,Ld);

char plain_bin[100];
permute(combine_dec,final_perm,64,plain_bin);

char plain_hex[20];
bin2hex(plain_bin,plain_hex);

printf("Decrypted Text: %s\n",plain_hex);

return 0;
}
```

Output:

```
Enter 16 Hex Plaintext: 123456ABCD132536
Enter 16 Hex Key: AABB09182736CCDD

Original Plaintext: 123456ABCD132536

Encrypted Cipher Text: C0B7A8D05F3A829C
Decrypted Text: 123456ABCD132536
```